

PARTIE 1 : FONCTIONS SCALAIRES

Objectif : Créer des fonctions qui retournent une **valeur unique**.

Exercice 1 : Calculer l'âge d'un client

Créer une fonction *calculer_age_client*(*id_client* *INTEGER*) qui retourne l'âge (en années) d'un client donné.

```
CREATE OR REPLACE FUNCTION calculer_age_client(id_client INTEGER)
RETURNS INTEGER AS $$
DECLARE
    v_date_naissance DATE;
    v_age INTEGER;
BEGIN
    -- Récupérer la date de naissance du client
    SELECT date_naissance INTO v_date_naissance
    FROM clients
    WHERE clients.id_client = calculer_age_client.id_client;
    IF NOT FOUND
    THEN RAISE EXCEPTION 'Client avec ID % non trouvé', id_client;
    END IF;

    -- Calculer l'âge
    v_age := EXTRACT(YEAR FROM AGE(CURRENT_DATE, v_date_naissance));

    RETURN v_age;

END;
$$ LANGUAGE plpgsql;

select calculer_age_client(60);
```

Exercice 2 : Vérifier si un client est majeur

Créer une fonction *est_majeur*(*id_client* *INTEGER*) qui retourne TRUE si le client est majeur (18 ans ou plus), FALSE sinon.

```
CREATE OR REPLACE FUNCTION est_majeur(id_client INTEGER)
RETURNS BOOLEAN AS $$
DECLARE
    v_majeur boolean=FALSE;
BEGIN
    IF calculer_age_client(id_client)>=18
    THEN v_majeur=TRUE::boolean;
    END IF;
    RETURN v_majeur;
END;
$$ LANGUAGE plpgsql;
select est_majeur(30);
```

Exercice 3 : Calculer le prix total d'une réservation

Créer une fonction *prix_total_reservation(id_reservation INTEGER)* qui retourne le montant total d'une réservation **en appliquant les promotions** associées au voyage.

```
CREATE OR REPLACE FUNCTION prix_total_reservation(id_resa INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_promo NUMERIC=0;
    v_tarif_voyage NUMERIC=0;
    v_nb_pers integer;
BEGIN
    -- recherche du prix et de la promo la meilleur du voyage
    select prix, coalesce((select pourcentage
                           from promotions join proposer using(id_promotion)
                           where v.id_voyage=id_voyage
                               and (date_depart >=date_debut
                                   and date_retour<=date_fin)
                           order by v.id_voyage, pourcentage desc limit 1),0)
    INTO v_tarif_voyage, v_promo
    from voyages v
    where id_voyage=(select id_voyage from reservations
                     where id_reservation=id_resa);

    -- recherche du nombre de personne
    v_nb_pers=(select nb_personnes from reservations
                where id_reservation=id_resa);

    --calcul du montant de la resa
    RETURN (v_tarif_voyage*(1-v_promo/100)*v_nb_pers)::decimal(10,2);
END;
$$ LANGUAGE plpgsql;

select prix_total_reservation(91);
```

Exercice 4 : mettre le montant des reservations

Créer une fonction *maj_resa()* qui modifie le montant total (colonne *montant* de *reservations*) d'une réservation **si celui-ci ne correspond pas** au montant calculé par la fonction *prix_total_reservation(id_reservation INTEGER)*.

Toutes les réservations seront vérifiées et mises à jour si nécessaire !

```
CREATE OR REPLACE FUNCTION maj_resa() RETURNS void AS $$
BEGIN
    update reservations R
    set montant = (select prix_total_reservation(R.id_reservation))
    where montant <> (select prix_total_reservation(R.id_reservation)) ;
END;
$$ LANGUAGE plpgsql;

select maj_resa();
```

Exercice 5 : Compter le nombre de réservations d'un client

Créer une fonction `nombre_reservations_client(id_client INTEGER)` qui retourne le nombre total de réservations (tous statuts confondus) pour un client.

```
CREATE OR REPLACE FUNCTION nombre_reservations_client(p_id_cli INTEGER)
RETURNS INTEGER AS $$
DECLARE
    v_nb INTEGER;
BEGIN
    -- Récupérer la date de naissance du client
    v_nb:=(select count(*) from reservations
           where id_client = p_id_cli);
    RETURN v_nb;

END;
$$ LANGUAGE plpgsql;

select nombre_reservations_client(1);
```

Exercice 6 : Calculer le montant moyen des réservations confirmées

Créer une fonction `montant_moyen_reservations_confirmées(id_client INTEGER)` qui retourne le **montant moyen** des réservations **confirmées** pour un client.

```
SELECT C.id_client,coalesce(AVG(montant),0)::numeric(10,2) as "Moyenne résa"
FROM CLIENTS C LEFT OUTER JOIN RESERVATIONS R ON R.ID_CLIENT=C.ID_CLIENT
GROUP BY C.id_client

CREATE OR REPLACE FUNCTION montant_moyen_reservations_confirmées(p_id_cli INTEGER)
RETURNS numeric(10,2) AS $$
DECLARE
    v_moyenne numeric(10,2) ;
BEGIN
    -- Récupérer la moyenne des resa du client
    SELECT coalesce(AVG(montant),0)::numeric(10,2) AS "Moyenne résa" INTO v_moyenne
    FROM clients C LEFT OUTER JOIN reservations R ON R.id_client=C.id_client
    WHERE C.id_client=p_id_cli
    GROUP BY C.id_client;

    IF NOT FOUND
    THEN RAISE EXCEPTION 'le client % n''existe pas',p_id_cli;
    END IF;

    RETURN v_moyenne;

END;
$$ LANGUAGE plpgsql;

SELECT montant_moyen_reservations_confirmées(64)
```

PARTIE 2 : FONCTIONS RETOURNANT DES JEUX DE RESULTATS (PLUS D'UNE LIGNE)

Objectif : Créer des fonctions qui **retournent plusieurs lignes**

Exercice 1 : Lister les voyages d'un guide

Créer une fonction *voyages_par_guide(id_guide INTEGER)* qui retourne **tous les voyages** (id_voyage, destination, date_depart, statut) gérés par un guide donné.

```
CREATE OR REPLACE FUNCTION voyages_par_guide(p_guide INTEGER)
returns SETOF RECORD AS $$
DECLARE
    v_guide record;
BEGIN
    Perform * from guides where id_guide=p_guide;
    if not found
        then
            raise exception 'le guide d''id % n''existe pas', p_guide;
        end if;

    for v_guide in select id_voyage, d.nom,pays, date_depart, statut
    from voyages join destinations d using(id_destination)
    where id_guide = p_guide
    loop
        return next v_guide;
    END LOOP;
    RETURN ;
END;
$$ LANGUAGE plpgsql;

select * from voyages_par_guide(6) as (id int, nom varchar, pays varchar,
date_dep date, statut varchar) ;
```

ou avec une table en sortie

```
CREATE OR REPLACE FUNCTION voyages_par_guide(p_guide INTEGER)
returns SETOF voyages AS $$
DECLARE
BEGIN
    Perform * from guides where id_guide=p_guide;
    if not found
        then
            raise exception 'le guide d''id % n''existe pas', p_guide;
        end if;

    return QUERY select voyages.*
    from voyages join destinations d using(id_destination)
    where id_guide = p_guide;

END;
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM voyages_par_guide(1)
```

Exercice 2 : Lister les destinations les plus populaires en nombre de réservations

Créer une fonction *destinations_populaires*(rang INTEGER DEFAULT 5) qui retourne les **destinations** (nom, pays, nombre de réservations) **les plus réservées correspondantes au rang passé en paramètre**, cad que si le rang est 3 on affichera les voyages qui sont au troisième rang en terme de nombre de réservation, le top 1 (rang 1) correspond au voyage qui on le plus de réservation !!!

```
CREATE OR REPLACE FUNCTION destinations_populaires(p_rang INTEGER DEFAULT 5)
returns SETOF RECORD AS $$
DECLARE
    v_voyage record;
BEGIN
    for v_voyage IN with
        nb_resa(idv, nbres) as (
            select id_voyage, count(*) over (partition by id_voyage)
            from reservations),
        rang_nb_resa (idv,nbres, rang) as (
            select distinct idv, nbres,
            dense_rank() over (order by nbres desc) as rang
            from nb_resa order by rang)

        select nom,pays,nbres::int
        from rang_nb_resa join voyages on idv=id_voyage
        join destinations using(id_destination) where rang = p_rang
    loop
        return next v_voyage;
    END LOOP;

    RETURN ;
END;
$$ LANGUAGE plpgsql;

select * from destinations_populaires(1) as (nom_voyage varchar, pays varchar,
nbresa int);
```

PARTIE 3 : FONCTIONS AVANCEES

Objectif : Utiliser des **curseurs**, des **boucles**, des **exceptions**

Exercice 3 : Lister les réservations non payées

Créer une fonction *reservations_non_payees()* qui retourne toutes les réservations dont le **montant total n'est pas couvert par les paiements** (id_reservation, id_client, montant_reservation, montant_paye, reste à payer).

```
CREATE OR REPLACE FUNCTION reservations_non_payees() returns SETOF RECORD AS $$
DECLARE
    v_resa record;
    v_sum_paye record;

    -- déclaration d'un curseur sur la somme des paiements par reservation CONFIRMEE
    curs_paielement CURSOR FOR
        select R.id_reservation,
               coalesce(sum(P.montant),0)::real AS MT_deja_paye,
               etat
        from paiements P right outer join reservations R
                                on R.id_reservation=P.id_reservation
        where statut = 'CONFIRMEE' and etat = 'VALIDE' or etat IS NULL
        group by R.id_reservation, etat
        having R.montant - coalesce(sum(P.montant),0) > 0;

BEGIN
    -- ouverture du curseur et parcours du curseur
    OPEN curs_paielement;
    LOOP
        FETCH curs_paielement INTO v_sum_paye;
        EXIT WHEN NOT FOUND;

        -- attention traitement différent si MT_deja_paye = 0,
        -- cad dire que id_reservation n'est pas dans la table paiement il reste donc
        -- tout à payer !!
        IF v_sum_paye.MT_deja_paye = 0
        THEN
            SELECT id_reservation, id_client, montant as Mt_resa,
                   0::real AS MT_deja_paye,
                   montant::real AS Reste_a_payer INTO v_resa
            FROM reservations R JOIN clients USING (id_client)
            WHERE id_reservation= v_sum_paye.id_reservation;

        ELSE
            SELECT id_reservation,id_client, R.montant AS Mt_resa,
                   v_sum_paye.MT_deja_paye::real,
                   (R.montant-v_sum_paye.MT_deja_paye)::real AS Reste_a_payer INTO v_resa
            FROM reservations R JOIN clients USING (id_client)
            WHERE v_sum_paye.MT_deja_paye < montant
            AND R.id_reservation= v_sum_paye.id_reservation;

        END IF;

        RETURN NEXT V_RESa;
    END LOOP;

    CLOSE curs_paielement;
END; $$ LANGUAGE plpgsql;

select * from reservations_non_payees() as (id_res int, id_cli int,
                                           mt_res numeric(10,2), mt_paye real, reste real ) ;
```

Exercice 2 : Appliquer une réduction à un voyage

Créer une fonction appliquer_reduction_voyage(id_voyage INTEGER, pourcentage NUMERIC) qui :

- **Met à jour** le prix du voyage en appliquant la réduction.
- **Retourne** le nouveau prix.

```
CREATE OR REPLACE FUNCTION appliquer_reduction_voyage(p_id_voyage INTEGER,  
pourcentage NUMERIC)  
RETURNS numeric(10,2) AS $$  
DECLARE  
    v_montant numeric(10,2) ;  
BEGIN  
  
    if NOT (pourcentage between 0 and 100)  
        then raise exception 'le pourcentage doit être compris entre 0 et 100';  
    end if;  
  
    -- Récupérer le montant du voyage  
    select prix into v_montant  
    from voyages  
    where id_voyage =p_id_voyage;  
  
    v_montant = v_montant *( 1 - (pourcentage/100) );  
  
    update voyages  
    set prix = v_montant  
    where id_voyage =p_id_voyage;  
  
    RETURN v_montant;  
  
END;  
$$ LANGUAGE plpgsql;  
  
select appliquer_reduction_voyage(1,50)
```

Exercice 3 : Annuler une réservation et rembourser

Créer une fonction annuler_reservation_et_rembourser(id_reservation INTEGER) qui :

1. **Met à jour** le statut de la réservation en 'ANNULEE'.
2. **Ajoute un paiement** de remboursement pour chaque paiement existant (même montant, etat = 'REMBOURSE').
3. **Retourne** TRUE si l'opération a réussi, FALSE sinon.

```
CREATE OR REPLACE FUNCTION annuler_reservation_et_rembourser(p_id_reservation INTEGER)
RETURNS boolean AS $$
DECLARE
    v_rec_paiement RECORD ;
    v_cur CURSOR(id int) IS SELECT id_paiement, montant FROM paiements WHERE
id_reservation = id and etat='VALIDE';
BEGIN

-- Vérifier que la réservation existe et n'est pas déjà annulée.
    if (select statut from reservations where id_reservation = p_id_reservation)
    = 'ANNULEE'
        then RETURN FALSE;
    end if;

-- met à jour le statut de la reservation
    update reservations
    set statut='ANNULEE'
    where id_reservation = p_id_reservation;

-- parcours du curseur
    OPEN v_cur(p_id_reservation);
    LOOP
        fetch v_cur into v_rec_paiement;
        exit when not found ;

        -- insère une ligne dans paiements
        INSERT INTO paiements VALUES (default, p_id_reservation, CURRENT_DATE,
v_rec_paiement.montant, 'VIREMENT', 'REMBOURSE');
    END LOOP;
    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;

select annuler_reservation_et_rembourser(1) ;
```

Exercice 4 : destination avec les meilleurs avis

Créer une fonction *meilleurs_destinations()* qui retourne toutes les informations sur les destinations avec le meilleurs avis.

```
CREATE OR REPLACE FUNCTION meilleurs_destinations() returns SETOF destinations
AS $$
DECLARE
    v_idDest int;
    v_destination destinations%ROWTYPE;

    -- déclaration d'un curseur sur la destination ave la meilleur moyenne des
notes
    curs_dest CURSOR FOR
        select id_destination
        from avis
        group by id_destination
        having avg(note)::decimal(4,2) >=ALL(
            select avg(note)::decimal(4,2)
            from avis
            group by id_destination);

BEGIN
    -- ouverture du curseur et parcours du curseur
OPEN curs_dest ;
LOOP
    fetch curs_dest INTO v_idDest;
    exit when NOT FOUND;

    select * INTO v_destination from destinations
    where id_destination =v_idDest;
    return next v_destination ;
END LOOP;
close curs_dest ;
END;
$$ LANGUAGE plpgsql;

select * from meilleurs_destinations() ;
```